Задания на практическую работу 05. Алгоритмы сортировки. Часть 4.

Номер Вашего варианта такой же, как в задании на практическую работу 02. Алгоритмы сортировки. Часть 1.

Решение – CLI программа на C#

1. Реализовать алгоритм поразрядной сортировке (radix sort) согласно условиям Вашего варианта (см ниже)

2. Реализовать алгоритм пирамидальной сортировки для простого целочисленного массива (LINQ и иные структуры данных кроме массивов в решении не использовать),
   в основной программе протестировать работу алгоритма:
   варианты 1, 2 – по возрастанию, восстановление кучи (heapify) рекурсивно
   варианты 3, 4, 9 – по возрастанию, восстановление кучи (heapify) итеративно
   варианты 5, 6, 10 – по убыванию, восстановление кучи (heapify) рекурсивно
   варианты 7, 8 – по убыванию, восстановление кучи (heapify) итеративно.

Варианты для задания 1.

Вариант 1. Сотрудник

```
class Employee {
    public string Name { get; set; }
    public string PersonnelNumber { get; set; } // формат "АБ-1234"
    public int Salary { get; set; }
    public string Department { get; set; }
}
```

Создать класс "Сотрудник", позволяющий обрабатывать в его объектах сведения о сотрудниках предприятия. Он должен включать данные: имя, табельный номер формата "АБ-1234" (2 буквы, дефис, 4 цифры), зарплата, отдел. Для табельного номера обеспечить контроль правильности хранимых в объекте данных (соответствие формату). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "табельный номер" алгоритмом **MSD поразрядной сортировки** как строку фиксированной длины. Вывести отсортированный массив на экран.
Без использования дополнительных структур данных и LINQ.

Вариант 2. Банковский счет

```
class BankAccount {
    public string OwnerName { get; set; }
    public long AccountNumber { get; set; }     // 10-значный номер счета
    public int Balance { get; set; }
    public int BankCode { get; set; }           // код банка
}
```

Создать класс "Банковский счет", позволяющий обрабатывать в его объектах сведения о счетах клиентов банка. Он должен включать данные: имя владельца, 10-значный номер счета, баланс, код банка. Для номера счета обеспечить контроль правильности хранимых в объекте данных (ровно 10

цифр). Создать не менее 2 конструкторов, метод ToString. В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла. Отсортировать этот массив по ключу "номер счета" алгоритмом **LSD поразрядной сортировки**. Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 3. Абонент

```
class Subscriber {
    public string Name { get; set; }
    public long PhoneNumber { get; set; }       // 11-значный номер телефона
    public int TariffId { get; set; }
    public int Debt { get; set; }
}
```

Задача: Создать класс "Абонент", позволяющий обрабатывать в его объектах сведения об абонентах телефонной компании. Он должен включать данные: имя абонента, 11-значный номер телефона, ID тарифа, задолженность. Для номера телефона обеспечить контроль правильности хранимых в объекте данных (ровно 11 цифр). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "номер телефона" алгоритмом **LSD поразрядной сортировки**. Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 4. Книга

```
class Book {
    public string Title { get; set; }
    public string Author { get; set; }
    public long ISBN { get; set; }              // 13-значный ISBN
    public int Year { get; set; }
}
```

Создать класс "Книга", позволяющий обрабатывать в его объектах сведения о книгах в библиотеке. Он должен включать данные: название, автор, 13-значный ISBN, год издания. Для ISBN обеспечить контроль правильности хранимых в объекте данных (ровно 13 цифр). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "ISBN" алгоритмом **LSD поразрядной сортировки**. Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 5. Гражданин

```
class Citizen {
    public string FullName { get; set; }
    public long PassportNumber { get; set; }    // 10-значный номер паспорта
    public int BirthYear { get; set; }
```

```
    public string Address { get; set; }
}
```

Создать класс "Гражданин", позволяющий обрабатывать в его объектах сведения о гражданах. Он должен включать данные: ФИО, 10-значный номер паспорта, год рождения, адрес. Для номера паспорта обеспечить контроль правильности хранимых в объекте данных (ровно 10 цифр). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "номер паспорта" алгоритмом **LSD поразрядной сортировки**.
Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 6. Посылка

```
class Parcel {
    public string Recipient { get; set; }
    public long TrackingNumber { get; set; }    // 14-значный трек-номер
    public int Weight { get; set; }
    public string Destination { get; set; }
}
```

Создать класс "Посылка", позволяющий обрабатывать в его объектах сведения о почтовых отправлениях. Он должен включать данные: получатель, 14-значный трек-номер, вес, пункт назначения. Для трек-номера обеспечить контроль правильности хранимых в объекте данных (ровно 14 цифр). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "трек-номер" алгоритмом **LSD поразрядной сортировки**.
Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 7. Товар

```
class Product {
    public string Name { get; set; }
    public long Barcode { get; set; }           // 13-значный штрих-код
    public int Price { get; set; }
    public string Category { get; set; }
}
```

Создать класс "Товар", позволяющий обрабатывать в его объектах сведения о товарах в магазине. Он должен включать данные: название, 13-значный штрих-код, цена, категория. Для штрих-кода обеспечить контроль правильности хранимых в объекте данных (ровно 13 цифр). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "штрих-код" алгоритмом **LSD поразрядной сортировки**.

Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 8. Студент

```
class Student {
    public string Name { get; set; }
    public long StudentId { get; set; }          // 8-значный номер студбилета
    public int Course { get; set; }
    public string Group { get; set; }
}
```

Создать класс "Студент", позволяющий обрабатывать в его объектах сведения о студентах вуза. Он должен включать данные: имя, 8-значный номер студенческого билета, курс, группа. Для номера студбилета обеспечить контроль правильности хранимых в объекте данных (ровно 8 цифр). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "номер студбилета" алгоритмом **LSD поразрядной сортировки**. Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 9. Автомобиль

```
class Car {
    public string Model { get; set; }
    public string LicensePlate { get; set; }    // формат "А123БВ" (6 символов)
    public int Year { get; set; }
    public int OwnerId { get; set; }
}
```

Создать класс "Автомобиль", позволяющий обрабатывать в его объектах сведения об автомобилях. Он должен включать данные: модель, номерной знак формата "А123БВ" (буква, 3 цифры, 2 буквы), год выпуска, ID владельца. Для номерного знака обеспечить контроль правильности хранимых в объекте данных (ровно 6 символов, соответствие формату). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "номерной знак" алгоритмом **MSD поразрядной сортировки** (старшие разряды вперед) как строку фиксированной длины. Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 11. Заказ

```
class Order {
    public string CustomerName { get; set; }
    public int OrderId { get; set; }            // 8-значный номер заказа
    public int Amount { get; set; }
    public string Status { get; set; }
```

```
}
```

Создать класс "Заказ", позволяющий обрабатывать в его объектах сведения о заказах в интернет-магазине. Он должен включать данные: имя клиента, 8-значный номер заказа, сумма, статус. Для номера заказа обеспечить контроль правильности хранимых в объекте данных (ровно 8 цифр). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "номер заказа" алгоритмом **LSD поразрядной сортировки**. Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 11. Деталь

```
class Detail {
    public string Name { get; set; }
    public int SerialNumber { get; set; }       // 8-значный серийный номер
    public int Weight { get; set; }
    public int BatchNumber { get; set; }        // номер партии
}
```

Создать класс "Деталь", позволяющий обрабатывать в его объектах сведения о деталях на заводе. Он должен включать данные: название, 8-значный серийный номер, вес, номер партии. Для серийного номера обеспечить контроль правильности хранимых в объекте данных (ровно 8 цифр). Создать не менее 2 конструкторов, метод ToString.
В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "серийный номер" алгоритмом **LSD поразрядной сортировки**. Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 12. Лекарство

```
class Medicine {
    public string Name { get; set; }
    public long RegistrationNumber { get; set; } // 12-значный рег. номер
    public int Price { get; set; }
    public string Manufacturer { get; set; }
}
```

Создать класс "Лекарство", позволяющий обрабатывать в его объектах сведения о лекарственных препаратах. Он должен включать данные: название, 12-значный регистрационный номер, цена, производитель. Для регистрационного номера обеспечить контроль правильности хранимых в объекте данных (ровно 12 цифр). Создать не менее 2 конструкторов, метод ToString. В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла.
Отсортировать этот массив по ключу "регистрационный номер" алгоритмом **LSD поразрядной сортировки**. Вывести отсортированный массив на экран. Без использования дополнительных структур данных и LINQ.

Вариант 13. Багаж

```csharp
class Luggage {
    public string OwnerName { get; set; }
    public int BaggageTag { get; set; }        // 6-значный номер багажной бирки
    public int FlightNumber { get; set; }
    public double Weight { get; set; }
}
```

Задача: Создать класс "Багаж", позволяющий обрабатывать в его объектах сведения о багаже авиапассажиров. Он должен включать данные: имя владельца, 6-значный номер багажной бирки, номер рейса, вес. Для номера багажной бирки обеспечить контроль правильности хранимых в объекте данных (6 цифр). Создать не менее 2 конструкторов, метод ToString. В основной CLI-программе сформировать массив из 30 объектов, прочитав их из заранее подготовленного текстового файла. Отсортировать этот массив по ключу "номер багажной бирки" алгоритмом **LSD поразрядной сортировки** (младшие разряды вперед). Вывести отсортированный массив на экран. В решении не использовать дополнительные структуры данных (списки, словари, множества) и LINQ.